

section 7.6: Error Handling -- Stderr and Exit

page 163

`stdout` and `stderr` are both predefined output streams; for our purposes, the only difference between them is that `stderr` is not likely to be redirected by the user, so the error messages printed to `stderr` will always appear on the screen, where they can be seen.

page 164

The cryptic note about ``a pattern-matching program" simply means that if you want to search the source code of a program for all the exit status values it can return, ``exit" might be an easier string to search for than ``return." (Every call to `exit` represents an exit from the program, but not every `return` statement does.)

The `feof` and `ferror` functions can be used to check for error conditions more carefully. In general, input routines (such as `getchar` and `getline`) return some special value to tell you that they couldn't read any more. Often, this value is `EOF`, reinforcing the notion that the only possible reason they couldn't read any more was because end-of-file had been reached. However, it's also possible that there was a read error, and you can call `feof` or `ferror` to determine whether this was the case. On the output side, though the output routines generally do return an error indication, few programs bother to check the return values from every call to functions such as `putchar` and `printf`. One way to check for output errors, without having to check the return value of every function, is to call `ferror` on the output stream (which might be `stdout`) at key points.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)